Enterprise-to-Agentic (E2A)

Architecture Framework

Implementation Playbook — Scaffold File, Class Contracts & Usage Guide

Framework repository: `github.com/subhamviky/e2a-framework`
Reference implementation: `github.com/subhamviky/order-to-cash-agentic-ai`
Scaffold file: `e2a_base.py — drop into any repo, inherit, override, run`
Document role: Next-level playbook extending the Master Abstraction Reference — concrete code, not just contracts

How This Document Relates to the Master Abstraction Reference

The Master Abstraction Reference defines WHAT the E2A Framework enforces (contracts, NFRs, access modifiers, parameter tables). This Playbook defines HOW to use it (the actual scaffold code, method bodies, configuration resolution chain, inheritance pattern, and end-to-end worked examples). Read the Master Reference first to understand the contract. Read this Playbook to implement it.

1. Purpose & Scope

This playbook provides everything a developer needs to implement the E2A Architecture Framework in a new or existing repository. It contains the complete, runnable scaffold file (e2a_base.py), detailed method signature specifications for all four primary abstract classes, configuration and environment variable resolution patterns, and a full worked example from inheritance through execution.

The document is structured in three parts:

- Part A — Class Contract Specifications: complete method signatures with config/kwargs for all classes
- Part B — Scaffold File: the ready-to-drop e2a_base.py with all abstract classes and docstrings
- Part C — Usage Guide: repo structure, inheritance, invocation, config/env integration, worked example

Single Rule

Every class has exactly one public entry point: run() for BaseAgent, execute() for BaseWorkflow and BaseToolService, retrieve() for BaseRAGPipeline. Application code calls only this public method. All protected methods (_validate_state, _build_messages, etc.) are called internally in a fixed sequence. Subclasses override only the protected methods they need.

Source Note: *Implementation examples and the categorical breakdown of cloud-native primitives in this playbook align with the AWS AI Ecosystem architectural map by Prashant Rathi.*

2. Class Contract Specifications

This section specifies the complete method contracts for each abstract class — access modifier, signature, accepted config/kwargs keys, and purpose. The config parameter and **kwargs together provide full flexibility: pass explicit values at call time, or rely on config dict defaults, or fall back to environment variables. The resolution order is always: kwargs override → config dict → environment variable → hardcoded default.

2.1 BaseAgent

agents/ — run() is the single public entry point

Constructor

Parameter	Type	Default	Required	Description
`state`	Dict[str, Any]	—	**Yes**	Agent state object. Must contain at minimum: query (str), intent (str), trace_id written by run().
`config`	Dict[str, Any]	`None`	**No**	Runtime configuration dict. If None, all values fall back to environment variables then hardcoded defaults.
`**kwargs`	Any	—	**No**	Optional per-call overrides. Values in kwargs take precedence over config dict and environment variables.

Method Contract Table

Access	Signature	Config / kwargs keys	Purpose
PUBLIC	`run(state, config=None, **kwargs) -> Dict`	Any config key; any kwarg	Template method. Calls all protected methods in fixed sequence. Assigns trace_id to state. Returns updated state.
PROTECTED	`_validate_state(state, config, **kwargs) -> bool`	schema, required_fields	Validate agent state pre-conditions. Return False to abort run().
PROTECTED	`_build_messages(state, config, **kwargs) -> list`	prompt_template, role	Construct [{role, content}] message list for LLM call.
PROTECTED	`_apply_policy(state, config, **kwargs) -> None`	retry_policy, circuit_breaker_config	Enforce governance: retries, circuit breakers, approval gates, redaction.
PRIVATE	`_llm_call(messages, config, **kwargs) -> dict`	model_name, max_tokens, temperature	Invoke LLM provider. Returns {text, metadata}. Provider-agnostic. Provider routing via config['model_id']
PROTECTED	`_evaluate_output(response, state, config, **kwargs) -> float`	min_confidence, groundedness_threshold	Score output quality [0.0-1.0]. Below min_confidence triggers _fallback().
PROTECTED	`_fallback(state, config, **kwargs) -> dict`	fallback_model, cache_key	Degraded response. No LLM call. Return safe static dict.
PROTECTED	`_handle_error(error, state, config, **kwargs) -> None`	alert_channel, retry_policy	Exception handler. Append to state['errors']. Do not re-raise.

Config / Environment Resolution — BaseAgent

config key	Environment variable	Hardcoded default	Used in step
`min_confidence`	`MIN_CONFIDENCE`	`0.85`	Step 5 — _evaluate_output() gate in run()
`model_name`	`LLM_MODEL_NAME`	`'default'`	Step 4 — _llm_call() provider selection
`max_tokens`	`MAX_TOKENS`	`2048`	Step 4 — _llm_call() token budget
`max_latency`	`MAX_LATENCY`	`2.0`	run() — latency SLO enforcement after Step 4

2.2 BaseWorkflow

orchestration/ — execute() is the single public entry point

Method Contract Table

Access	Signature	Config / kwargs keys	Purpose
PUBLIC	`execute(state, config=None, **kwargs) -> Dict`	workflow_id, routing_strategy	Template method. Builds workflow, validates, routes to agent, calls agent.run().
PROTECTED	`_build_workflow(config, **kwargs) -> Any`	dag_definition, node_configs	Define LangGraph StateGraph or equivalent. Return compiled graph.
PROTECTED	`_validate_workflow(workflow, config, **kwargs) -> bool`	schema, validation_rules	Check graph structure. Return False to abort execute().
PROTECTED	`_get_agent(intent, config, **kwargs) -> BaseAgent`	routing_strategy, agent_registry	Return instantiated agent for given intent string. @abstractmethod — enforced.
PROTECTED	`_handle_error(error, state, config, **kwargs) -> None`	alert_channel, retry_policy	Workflow-level exception handler. Append to state['errors'].

2.3 BaseRAGPipeline

rag/ — retrieve() is the single public entry point

Constructor

Parameter	Type	Default	Required	Description
`query`	str	—	**Yes**	Input query string passed to _chunk_text() and _embed_text().
`config`	Dict[str, Any]	`None`	**No**	Pipeline configuration. All keys fall back to env vars then defaults if not provided.
`**kwargs`	Any	—	**No**	Per-call overrides: top_k, rerank_strategy, etc.

Method Contract Table

Access	Signature	Config / kwargs keys	Purpose
PUBLIC	`retrieve(query, config=None, **kwargs) -> str`	top_k, rerank_strategy, faithfulness_threshold	Template method. Full RAG cycle: chunk → embed → index → search → rerank → generate → evaluate. Returns answer string.
PROTECTED	`_chunk_text(text, config, **kwargs) -> list[str]`	chunk_size, overlap	Split text into chunks. Returns list of strings.
PROTECTED	`_embed_text(text, config, **kwargs) -> list[float]`	embedding_model	Generate dense vector embedding. Provider-agnostic.
PROTECTED	`_index_document(doc_id, text, metadata, config, **kwargs) -> None`	index_name, ttl	Persist chunk + embedding to vector store.
PROTECTED	`_search_index(query_vector, config, **kwargs) -> list[dict]`	top_k, filter	Execute vector search. Returns list of {doc_id, content, score}.

PROTECTED	`_rerank(results, config, **kwargs) -> list[dict]`	rerank_strategy, reranker_model	Reorder results by cross-encoder or provider score.
PROTECTED	`_generate_answer(results, config, **kwargs) -> str`	answer_template, model_name	LLM generation from retrieved contexts. Returns answer string.
PROTECTED	`_evaluate_answer(answer, config, **kwargs) -> float`	groundedness_threshold	RAGAS or equivalent faithfulness score. Below threshold: retrieve() raises.

Config / Environment Resolution — BaseRAGPipeline

config key	Environment variable	Default	Used in step
`faithfulness_threshold`	`FAITHFULNESS_THRESHOLD`	`0.85`	retrieve() — gate after _evaluate_answer()
`top_k`	`RAG_TOP_K`	`5`	_search_index() — candidate document count
`embedding_model`	`EMBEDDING_MODEL`	`'default-embed'`	_embed_text() — provider model selection
`index_name`	`RAG_INDEX_NAME`	`'rag-index'`	_index_document(), _search_index()

2.4 BaseToolService

tools/ — execute() is the single public entry point (async)

Method Contract Table

Access	Signature	Config / kwargs keys	Purpose
PUBLIC	`async execute(payload, config=None, **kwargs) -> dict`	timeout, retries, auth_token	Template method (async). Validates → gets endpoint → builds payload → HTTP POST. Returns result dict.
PROTECTED	`get_endpoint(config, **kwargs) -> str`	base_url, service_name	Return HTTP URL for this tool. Read from config['tool_base_url'].
PROTECTED	`_build_payload(payload, config, **kwargs) -> dict`	schema, enrichments	Transform input dict to tool API request schema.
PROTECTED	`_validate_input(payload, config, **kwargs) -> bool`	schema, required_fields	Domain-specific validation. Return False to raise ValueError in execute().
PRIVATE	`async _http_post(endpoint, body, config, **kwargs) -> dict`	timeout, retries, headers	Perform HTTP POST. Handles retries and timeout internally. Async. HTTP mechanics via config['timeout', 'retries', 'auth_token']

Config / Environment Resolution — BaseToolService

config key	Environment variable	Default	Used in step
`timeout`	`TOOL_TIMEOUT`	`5.0`	_http_post() — HTTP session timeout
`retries`	`TOOL_RETRIES`	`3`	execute() — retry loop count
`tool_base_url`	`TOOL_BASE_URL`	`'http://localhost'`	get_endpoint() — base URL prefix
`auth_token`	`TOOL_AUTH_TOKEN`	`None`	_http_post() — Authorization header

2.5 Foundation Classes

Interface pattern — BaseInfraProvisioner · BaseObservability · BasePipeline · BaseGovernanceFramework

Interface Pattern — No Shared Implementation

Foundation classes are Interfaces (Python Protocol or pure ABC with no __init__ state). They carry no shared implementations. The abstract class enforces the contract; the project-specific subclass owns the complete implementation. Every method accepts config=None and **kwargs so the same signature pattern is consistent across all E2A classes.

Access	Class / Method	Config / kwargs keys	Purpose
INTERFACE	`BaseInfraProvisioner`		
INTERFACE	`_define_network(config, **kwargs)`	cidr_block, region, az_count	Define VPC, subnets, security groups.
INTERFACE	`_define_compute(config, **kwargs)`	instance_type, desired_count	Define ECS/GKE/Cloud Run compute resources.
INTERFACE	`_define_storage(config, **kwargs)`	table_name, ttl, billing_mode	Define DynamoDB/Firestore/Spanner tables.
INTERFACE	`_define_secrets(config, **kwargs)`	secret_name, rotation_days	Define Secrets Manager/Key Vault resources.
INTERFACE	`_apply_infra(config, **kwargs)`	dry_run, auto_approve	Execute terraform apply/CDK deploy/Pulumi up.
INTERFACE	`BaseObservability`		
INTERFACE	`_collect_metrics(config, **kwargs)`	namespace, dimensions	Emit custom metrics to CloudWatch/Datadog/GCP Monitoring.
INTERFACE	`_collect_traces(config, **kwargs)`	trace_id, service_name	Emit distributed traces to X-Ray/Jaeger/Cloud Trace.
INTERFACE	`_collect_logs(config, **kwargs)`	log_level, correlation_id	Structured log shipping to CloudWatch/Datadog/GCP Logging.
INTERFACE	`_enforce_slo(latency, avail, config, **kwargs)`	slo_targets, alert_channel	Evaluate SLOs, emit violation events, trigger alerts.
INTERFACE	`BasePipeline`		
INTERFACE	`_run_tests(config, **kwargs)`	coverage_threshold, test_markers	Unit + integration test gate.
INTERFACE	`_run_rag_eval(config, **kwargs)`	faithfulness_threshold, eval_dataset	RAGAS evaluation gate. Block deploy if below threshold.
INTERFACE	`_build_artifacts(config, **kwargs)`	image_tag, registry_url	Docker build, ECR push, Trivy scan.
INTERFACE	`_deploy_service(config, **kwargs)`	cluster, service_name, image_tag	ECS rolling deploy / Cloud Run / Kubernetes apply.
INTERFACE	`_enforce_policy(config, **kwargs)`	policy_path, opa_bundle	YAML policy validation / OPA check / Bandit SAST.
INTERFACE	`BaseGovernanceFramework`		
INTERFACE	`_apply_policy(payload, config, **kwargs)`	policy_file, redaction_rules	Policy enforcement and PII redaction.
INTERFACE	`_track_costs(cost, config, **kwargs)`	metric_namespace, budget_limit	FinOps cost recording per workflow.

INTERFACE	_enforce_slo(metrics, config, **kwargs)	slo_targets, freeze_on_breach	Error budget tracking, deployment freeze on breach.
INTERFACE	_circuit_breaker(failures, config, **kwargs)	fail_max, reset_timeout	Circuit breaker state transitions + metric emission.

3. Scaffold File — e2a_base.py

Drop this file into any repository as the foundation layer. All E2A abstract classes are defined here with full docstrings, @abstractmethod decorators, config/env resolution examples, and trace_id generation. Developers import and subclass these without modifying this file.

File placement
Place e2a_base.py at src/e2a_base.py or framework/e2a_base.py. This becomes the architectural foundation module. All concrete agent, workflow, RAG, and tool classes import from it. The file must never be modified once placed — create subclasses instead.

e2a_base.py — Complete Source

```
e2a_base.py — E2A Architecture Framework Scaffold
Drop into src/ or framework/. Import and subclass. Never modify.
github.com/subhamviky/e2a-framework

import os
import uuid
import time
import logging
from abc import ABC, abstractmethod
from typing import Any, Dict, List

logging.basicConfig(level=logging.INFO)



================================================
BaseAgent
================================================
class BaseAgent(ABC):
    def run(self, state: Dict[str, Any],
            config: Dict[str, Any] = None,
            **kwargs) -> Dict[str, Any]:
        """
        Public entry point for agent execution.
        Parameters:
            state  : Agent state object (dict).
            config : Runtime configuration (optional).
            kwargs : Per-call overrides (min_confidence, max_latency, etc.).
        Resolution: kwargs > config dict > env variable > hardcoded default.
        Example: config.get('min_confidence',
                    float(os.getenv('MIN_CONFIDENCE', 0.85)))
        """
        config = config or {}
        trace_id = str(uuid.uuid4())
        state['trace_id'] = trace_id
        start = time.time()

        try:
            self._validate_state(state, config, **kwargs)
```

```
            messages = self._build_messages(state, config, **kwargs)
            self._apply_policy(state, config, **kwargs)
            response = self.__llm_call(messages, config, **kwargs)
            confidence = self._evaluate_output(response, state,
                                               config, **kwargs)

            min_conf = kwargs.get(
                'min_confidence',
                config.get('min_confidence',
                    float(os.getenv('MIN_CONFIDENCE', 0.85))))

            if confidence < min_conf:
                self._fallback(state, config, **kwargs)

            state['response'] = response
        except Exception as e:
            self._handle_error(e, state, config, **kwargs)

        return state

    @abstractmethod
    def _validate_state(self, state, config=None, **kwargs):
        """Validate agent state.
        kwargs: schema, required_fields
        Defaults from config/env if not passed."""
        pass

    @abstractmethod
    def _build_messages(self, state, config=None, **kwargs):
        """Construct LLM messages.
        kwargs: prompt_template, role
        Defaults applied if not passed."""
        pass

    @abstractmethod
    def _apply_policy(self, state, config=None, **kwargs):
        """Apply governance rules.
        kwargs: retry_policy, circuit_breaker_config
        Config/env values used if not passed."""
        pass

    # PRIVATE — base class owns this entirely
    def __llm_call(self, messages, config=None, **kwargs):
            """
            Provider-agnostic LLM dispatch.
            Reads model_id from kwargs > config > env.
            Routes to correct provider internally.
            Subclasses never override this — they configure via model_id.
            """
            model_id = kwargs.get(
            'model_id',
            config.get('model_id',
            os.getenv('LLM_MODEL_ID', 'default')))

            # Route by prefix — subclasses configure, never override
            if 'anthropic' in model_id or 'amazon' in model_id:
                   return self.__call_bedrock(messages, model_id, config, **kwargs)
            elif 'gemini' in model_id or 'text-bison' in model_id:
                   return self.__call_vertex(messages, model_id, config, **kwargs)
            elif 'gpt' in model_id:
                   return self.__call_openai(messages, model_id, config, **kwargs)
            else:
                   raise ValueError(f"Unknown model_id: {model_id}")

    @abstractmethod
    def _evaluate_output(self, response, state,
                         config=None, **kwargs):
```

```
        """Evaluate output quality. Return float [0.0-1.0].
        kwargs: min_confidence, groundedness_threshold
        Defaults applied if not passed."""
        pass

    @abstractmethod
    def _fallback(self, state, config=None, **kwargs):
        """Fallback strategy (secondary model, cache).
        kwargs: fallback_model, cache_key
        Config/env values used if not passed."""
        pass

    @abstractmethod
    def _handle_error(self, error, state, config=None, **kwargs):
        """Error handling. Append to state['errors'].
        kwargs: alert_channel, retry_policy
        Defaults applied if not passed."""
        pass


# ================================================
# BaseWorkflow
# ================================================
class BaseWorkflow(ABC):
    def execute(self, state: Dict[str, Any],
                config: Dict[str, Any] = None,
                **kwargs) -> Dict[str, Any]:
        """
        Public entry point for workflow execution.
        Parameters:
            state  : Agent state object.
            config : Runtime configuration (optional).
            kwargs : Per-call overrides (workflow_id, routing_strategy).
        """
        config = config or {}
        try:
            workflow = self._build_workflow(config, **kwargs)
            self._validate_workflow(workflow, config, **kwargs)
            agent = self._get_agent(
                state.get('intent'), config, **kwargs)
            return agent.run(state, config, **kwargs)
        except Exception as e:
            self._handle_error(e, state, config, **kwargs)
            return state

    @abstractmethod
    def _build_workflow(self, config=None, **kwargs):
        """Define workflow graph.
        kwargs: dag_definition, node_configs"""
        pass

    @abstractmethod
    def _validate_workflow(self, workflow,
                           config=None, **kwargs):
        """Validate workflow correctness.
        kwargs: schema, validation_rules"""
        pass

    @abstractmethod
    def _get_agent(self, intent, config=None, **kwargs):
        """Route to agent based on intent.
        kwargs: routing_strategy"""
        pass

    @abstractmethod
    def _handle_error(self, error, state,
                      config=None, **kwargs):
```

```
        """Workflow-level error handling.
        kwargs: alert_channel, retry_policy"""
        pass


# ================================================
# BaseRAGPipeline
# ================================================
class BaseRAGPipeline(ABC):
    def retrieve(self, query: str,
                 config: Dict[str, Any] = None,
                 **kwargs) -> str:
        """
        Public entry point for retrieval.
        Parameters:
            query  : Input query string.
            config : Runtime configuration (optional).
            kwargs : Per-call overrides (top_k, rerank_strategy).
        """
        config = config or {}
        try:
            chunks = self._chunk_text(query, config, **kwargs)
            embeddings = [self._embed_text(c, config, **kwargs)
                          for c in chunks]
            self._index_document('query', query, {},
                                 config, **kwargs)
            results = self._search_index(embeddings,
                                         config, **kwargs)
            reranked = self._rerank(results, config, **kwargs)
            answer = self._generate_answer(reranked,
                                           config, **kwargs)
            score = self._evaluate_answer(answer,
                                          config, **kwargs)

            threshold = kwargs.get(
                'faithfulness_threshold',
                config.get('faithfulness_threshold',
                    float(os.getenv(
                        'FAITHFULNESS_THRESHOLD', 0.85))))

            if score < threshold:
                raise ValueError(
                    'Answer faithfulness below threshold')

            return answer
        except Exception as e:
            logging.error(f'RAG retrieve failed: {e}')
            return ''

    @abstractmethod
    def _chunk_text(self, text, config=None, **kwargs):
        """Chunk text. kwargs: chunk_size, overlap."""
        pass

    @abstractmethod
    def _embed_text(self, text, config=None, **kwargs):
        """Generate embeddings. kwargs: embedding_model."""
        pass

    @abstractmethod
    def _index_document(self, doc_id, text, metadata,
                        config=None, **kwargs):
        """Index document. kwargs: index_name, ttl."""
        pass

    @abstractmethod
    def _search_index(self, query_vector,
```

```
                      config=None, **kwargs):
        """Search index. kwargs: top_k, filter."""
        pass

    @abstractmethod
    def _rerank(self, results, config=None, **kwargs):
        """Rerank results. kwargs: rerank_strategy."""
        pass

    @abstractmethod
    def _generate_answer(self, results,
                         config=None, **kwargs):
        """Generate final answer. kwargs: answer_template."""
        pass

    @abstractmethod
    def _evaluate_answer(self, answer,
                         config=None, **kwargs):
        """Evaluate answer quality.
        kwargs: groundedness_threshold"""
        pass



================================================
BaseToolService
================================================
class BaseToolService(ABC):
    async def execute(self, payload: Dict[str, Any],
                      config: Dict[str, Any] = None,
                      **kwargs) -> Dict[str, Any]:
        """
        Public entry point for tool execution (async).
        Parameters:
            payload : Input payload dict.
            config  : Runtime configuration (optional).
            kwargs  : Per-call overrides (timeout, retries).
        """
        config = config or {}
        if not self._validate_input(payload, config, **kwargs):
            raise ValueError('Invalid input')
        endpoint = self.get_endpoint(config, **kwargs)
        body = self._build_payload(payload, config, **kwargs)
        result = await self.__http_post(endpoint, body,
                                        config, **kwargs)
        return result

    @abstractmethod
    def get_endpoint(self, config=None, **kwargs):
        """Return endpoint URL.
        kwargs: base_url, service_name"""
        pass

    @abstractmethod
    def _build_payload(self, payload,
                       config=None, **kwargs):
        """Build request payload.
        kwargs: schema, enrichments"""
        pass

    @abstractmethod
    def _validate_input(self, payload,
                        config=None, **kwargs):
        """Validate input.
        kwargs: schema, required_fields"""
        pass

    # PRIVATE — framework owns HTTP mechanics
```

```
    async def __http_post(self, endpoint, body, config=None, **kwargs):
        """
        Async HTTP POST with retry, timeout, auth.
        Fully config-driven. Subclasses never override.
        """
        timeout  = kwargs.get('timeout',  config.get('timeout', float(os.getenv('TOOL_TIMEOUT',    5.0))))
        retries  = kwargs.get('retries',  config.get('retries',  int(os.getenv('TOOL_RETRIES', 3))))
        token    = kwargs.get('auth_token', config.get('auth_token', os.getenv('TOOL_AUTH_TOKEN')))
        headers  = {'Authorization': f'Bearer {token}'} if token else {}
        # ... retry loop, aiohttp call, latency check



# ================================================
# Foundation Classes (Interface pattern)
# No shared implementations. Subclass owns all logic.
# ================================================
class BaseInfraProvisioner(ABC):
    @abstractmethod
    def _define_network(self, config=None, **kwargs):
        """Define VPC, subnets.
        kwargs: cidr_block, region, az_count"""
        pass
    @abstractmethod
    def _define_compute(self, config=None, **kwargs):
        """Define compute resources.
        kwargs: instance_type, desired_count"""
        pass
    @abstractmethod
    def _define_storage(self, config=None, **kwargs):
        """Define storage/databases.
        kwargs: table_name, ttl, billing_mode"""
        pass
    @abstractmethod
    def _define_secrets(self, config=None, **kwargs):
        """Define secret management.
        kwargs: secret_name, rotation_days"""
        pass
    @abstractmethod
    def _apply_infra(self, config=None, **kwargs):
        """Execute provisioning.
        kwargs: dry_run, auto_approve"""
        pass


class BaseObservability(ABC):
    @abstractmethod
    def _collect_metrics(self, config=None, **kwargs):
        """Emit custom metrics.
        kwargs: namespace, dimensions"""
        pass
    @abstractmethod
    def _collect_traces(self, config=None, **kwargs):
        """Emit distributed traces.
        kwargs: trace_id, service_name"""
        pass
    @abstractmethod
    def _collect_logs(self, config=None, **kwargs):
        """Ship structured logs.
        kwargs: log_level, correlation_id"""
        pass
    @abstractmethod
    def _enforce_slo(self, latency, avail,
                     config=None, **kwargs):
        """Evaluate SLOs, trigger alerts.
        kwargs: slo_targets, alert_channel"""
        pass


class BasePipeline(ABC):
```

```
    @abstractmethod
    def _run_tests(self, config=None, **kwargs):
        """Unit + integration test gate.
        kwargs: coverage_threshold, test_markers"""
        pass
    @abstractmethod
    def _run_rag_eval(self, config=None, **kwargs):
        """RAG faithfulness gate.
        kwargs: faithfulness_threshold, eval_dataset"""
        pass
    @abstractmethod
    def _build_artifacts(self, config=None, **kwargs):
        """Build, package, scan.
        kwargs: image_tag, registry_url"""
        pass
    @abstractmethod
    def _deploy_service(self, config=None, **kwargs):
        """Deploy service.
        kwargs: cluster, service_name, image_tag"""
        pass
    @abstractmethod
    def _enforce_policy(self, config=None, **kwargs):
        """Policy compliance gate.
        kwargs: policy_path, opa_bundle"""
        pass


class BaseGovernanceFramework(ABC):
    @abstractmethod
    def _apply_policy(self, payload,
                      config=None, **kwargs):
        """Policy enforcement + PII redaction.
        kwargs: policy_file, redaction_rules"""
        pass
    @abstractmethod
    def _track_costs(self, cost, config=None, **kwargs):
        """FinOps cost recording.
        kwargs: metric_namespace, budget_limit"""
        pass
    @abstractmethod
    def _enforce_slo(self, metrics,
                     config=None, **kwargs):
        """SLO evaluation + error budget.
        kwargs: slo_targets, freeze_on_breach"""
        pass
    @abstractmethod
    def _circuit_breaker(self, failures,
                         config=None, **kwargs):
        """Circuit breaker state management.
        kwargs: fail_max, reset_timeout"""
        pass
```

4. Usage Guide

4.1 Repo Structure

After placing e2a_base.py, the recommended repo structure is:

```
repo-root/
├── src/
│   ├── e2a_base.py          # E2A scaffold — never modify
│   ├── agents/              # Concrete agent classes
```

```
│   │   ├── router_agent.py
│   │   ├── refund_agent.py
│   │   └── knowledge_agent.py
│   ├── workflows/           # Workflow implementations
│   │   └── o2c_workflow.py
│   ├── rag/                 # RAG pipeline implementations
│   │   └── opensearch_pipeline.py
│   └── tools/               # Tool service implementations
│       ├── create_order_tool.py
│       └── check_stock_tool.py
├── tests/
├── .env
└── requirements.txt
```

4.2 Creating an Inherited Class

Subclass the abstract base class and override only the protected methods relevant to your domain. The public method (run, execute, retrieve) is never overridden — it is inherited and used directly.

Example: RefundAgent inheriting from BaseAgent

```python
src/agents/refund_agent.py
from src.e2a_base import BaseAgent

class RefundAgent(BaseAgent):
    agent_name = 'RefundAgent'  # governance key — rename-safe

    def _validate_state(self, state, config=None, **kwargs):
        # Required field: order_id
        return 'order_id' in state

    def _build_messages(self, state, config=None, **kwargs):
        # Reads prompt_template from kwargs or config or default
        template = kwargs.get(
            'prompt_template',
            config.get('prompt_template',
                'Process refund for order {order_id}'))
        content = template.format(
            order_id=state['order_id'])
        return [
            {'role': 'system',
             'content': 'You are a refund specialist.'},
            {'role': 'user', 'content': content}
        ]

    def _apply_policy(self, state, config=None, **kwargs):
        # Enforce refund policy: amount must not exceed limit
        limit = kwargs.get(
            'refund_limit',
            config.get('refund_limit',
                float(os.getenv('REFUND_LIMIT', 500.0))))
        if state.get('amount', 0) > limit:
            raise ValueError(
                f'Refund exceeds limit: {limit}')

    def _evaluate_output(self, response, state,
                         config=None, **kwargs):
        # Simple confidence: always 1.0 for deterministic refunds
        return 1.0

    # _apply_policy, _fallback, _handle_error: use default pass
    # _llm_call: PRIVATE — configured via config['model_id'], never override
```

4.3 Invoking the Public Entry Point

Every class has a single public method. Application code calls only this method. Protected methods are never called directly.

Pattern 1: Direct invocation

```
Minimal call — config and kwargs both optional
agent = RefundAgent()
state = {'order_id': 'ORD-123', 'amount': 150.0}
result = agent.run(state)
print(result['response'])
```

```
With config dict
config = {
    'min_confidence': 0.9,
    'model_name': 'anthropic.claude-3-sonnet-v1',
    'refund_limit': 1000.0
}
result = agent.run(state, config=config)
```

```
With per-call kwargs override (highest priority)
result = agent.run(
    state,
    config=config,
    min_confidence=0.95,   # overrides config value
    model_name='claude-3-haiku'  # cheaper model for this call
)
```

Pattern 2: LangGraph node

```
LangGraph node: same call, no wrapper needed
def refund_node(state: dict) -> dict:
    agent = RefundAgent()
    return agent.run(state, config=AGENT_CONFIG)
```

Pattern 3: FastAPI endpoint

```
FastAPI: public method maps directly to endpoint
@router.post('/api/v1/refund', response_model=AgentState)
async def process_refund(request: RefundRequest,
                         settings = Depends(get_settings)):
    agent = RefundAgent()
    return agent.run(
        state=request.to_state(),
        config=settings.agent_config
    )
```

4.4 Config and Environment Variable Integration

The resolution chain is consistent across all E2A classes:

```
# Resolution order (highest to lowest priority):
# 1. kwargs passed at call time
# 2. config dict passed at call time
# 3. Environment variable
# 4. Hardcoded default in scaffold

# Inside any protected method:
value = kwargs.get(
    'my_param',
```

```
    config.get('my_param',
        type_cast(os.getenv('MY_PARAM', 'default'))))
```

Environment variables for all classes

```
Agent
export MIN_CONFIDENCE=0.85
export MAX_TOKENS=2048
export LLM_MODEL_NAME='anthropic.claude-3-sonnet-v1'
export MAX_LATENCY=2.0
```

```
RAG Pipeline
export FAITHFULNESS_THRESHOLD=0.85
export RAG_TOP_K=5
export EMBEDDING_MODEL='amazon.titan-embed-text-v1'
export RAG_INDEX_NAME='my-knowledge-index'
```

```
Tool Service
export TOOL_BASE_URL='https://api.myservice.com'
export TOOL_TIMEOUT=5.0
export TOOL_RETRIES=3
export TOOL_AUTH_TOKEN='Bearer eyJ...'
```

4.5 Guidance: Which Methods to Override

Protected Method	Override?	When and why	Default (if not overridden)
_validate_state	**Recommended**	Almost always: every agent has different required fields in state.	pass — no validation
_build_messages	**Yes**	Always: every agent needs a domain-specific system prompt and user message.	pass — no messages
_apply_policy	**If governed**	When the agent has domain-specific approval gates, tool deny-lists, or must-cite rules.	pass — no policy
_evaluate_output	**Yes**	Always: define what 'quality' means for this agent (RAGAS, keyword check, rule-based, etc.).	pass — returns None
_fallback	**Recommended**	For production: define a safe degraded response so run() never returns an empty state.	pass — no fallback
_handle_error	**Recommended**	For production: log the error, append to state['errors'], emit an alert.	pass — silent

4.6 Multi-Class End-to-End Example

This example shows a complete OrderOpsAgent that inherits from BaseAgent and uses a RAG pipeline (inheriting from BaseRAGPipeline) to ground its response, then calls a tool service (inheriting from BaseToolService).

OrderOpsAgent — agent + RAG + tool in one workflow

```
src/agents/order_ops_agent.py
from src.e2a_base import BaseAgent
from src.rag.opensearch_pipeline import OpenSearchRAGPipeline
from src.tools.create_order_tool import CreateOrderTool

class OrderOpsAgent(BaseAgent):
    agent_name = 'OrderOpsAgent'
```

```
    def _validate_state(self, state, config=None, **kwargs):
        return all(k in state for k in
                   ['query', 'user_id', 'intent'])

    def _build_messages(self, state, config=None, **kwargs):
        # Retrieve context before building messages
        pipeline = OpenSearchRAGPipeline(config=config)
        context = pipeline.retrieve(
            query=state['query'],
            config=config,
            top_k=kwargs.get('top_k', 5)
        )
        return [
            {'role': 'system',
             'content': f'Use this context: {context}'},
            {'role': 'user',
             'content': state['query']}
        ]

    def _apply_policy(self, state, config=None, **kwargs):
        # Call tool after LLM decides action
        if state.get('action') == 'create_order':
            import asyncio
            tool = CreateOrderTool(config=config)
            # Idempotency on by default for write tools
            result = asyncio.run(
                tool.execute(state['order_params'],
                             config=config))
            state['tool_result'] = result

    def _evaluate_output(self, response, state,
                         config=None, **kwargs):
        return 1.0 if response.get('text') else 0.0

# Invocation — one public call, everything else is internal
agent = OrderOpsAgent()
result = agent.run(
    state={
        'query': 'Create order for 100 units of SKU-001',
        'user_id': 'user_123',
        'intent': 'ORDER_OPS'
    },
    config={
        'min_confidence': 0.85,
        'top_k': 3,
        'tool_base_url': 'https://tools.myservice.com'
    }
)
```